ChatGPT

KiCad EAGLE Agent Tools

You

261106-02 hi Lyra, pls for schematic/pcb design I today use KiCad and I already tested for some guy in gg retro-comp to use also grok, but of course, I primarily expect that KiCad IS installed and that agentic harness will be happier to use say kicad-cli (confirmed here) - Andrew on gg wants to generate s-expressions, to generate kicad sch and pcb files WITHOUT the kicad-cli, this must be FAR HARDER and also expensive for tokens, ya? ... I dont understand now the reason, KiCad is installable anywhere and in fact imho necessary for immediate interactive checking/testing ... not sure, if he is not using only online version of grok or so ... must ask, I want to help guys to use that all well (because there are still sad rumors about "vibe sloping", but using local app (gui or cli, often similar) with access to filesystem, rights to use "tools" as kicad-cli, this is all just easy and fun, in interactive mode, so NOT "one shoot vibed" entire product without any human touch ... this is common misconception and wanderers are trying that, sure ... there are REAL engineers, who may do 100x more work with agentic assistants, and have great fun while doing so ... thats the motivation here;

another thing for other guy is checking possibility to use also "eagle" , not sure about versions, but thing is the same, has it some eagle-cli or API to be used as "skill" by agentic harness?? I also know ppl who are used to use eagle and nothing else; job of agentic tools is to learn to use ANY tools, prefferable CLI as its easier/cheaper for them and more precise and of course "headless", so not necessary to launch visible GUI and play there with menus and pointer ...grrr, sure

Worked for 1m 13s

Take your curiosity further
Scan to get the ChatGPT app

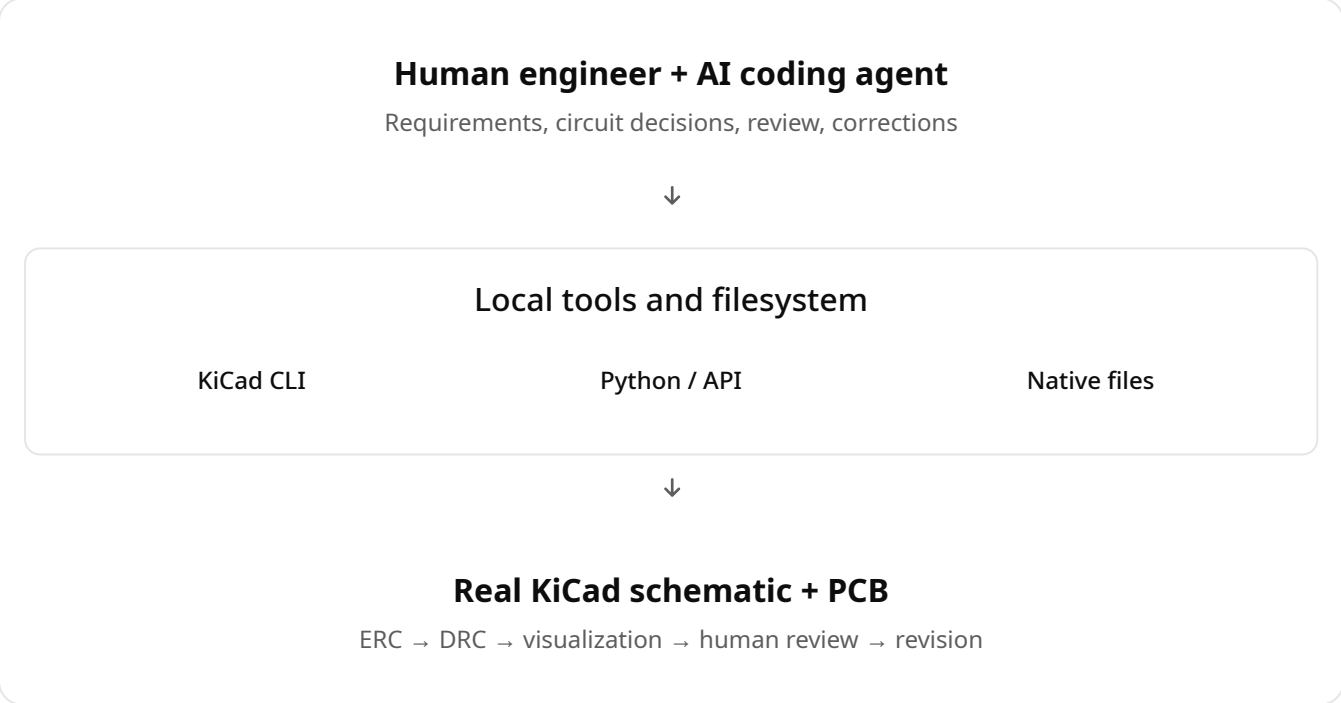
ChatGPT

Hi! :-) This is actually a very interesting subject, especially for your GG retro-computing guys. And I agree with your central point: **an engineering AI assistant should use the engineering software already available, rather than try to reinvent the entire application through text generation.**

But there is one important technical distinction, and some rather interesting news about both KiCad and EAGLE in 2026.

1. KiCad: yes, but `kicad-cli` is only part of the answer

The ideal arrangement is:



The distinction is that `kicad-cli` **primarily validates, exports and processes designs. It isn't a general-purpose schematic/PCB creation command interface.**

For example, the current KiCad CLI can run schematic ERC, PCB DRC, export netlists, PDF/SVG drawings, manufacturing files and more. But creating or modifying the actual circuit topology requires a different mechanism: a programming API, scripting, or manipulating the native KiCad files. KiCad +1

Take your curiosity further
Scan to get the ChatGPT app

So Andrew generating KiCad S-expressions isn't necessarily doing something fundamentally wrong. Those are the native design files. The inefficient part would be attempting to generate an entire complicated file repeatedly, without using KiCad itself for validation and rendering.

And there is something even better coming.

2. KiCad's new programming API is especially interesting

As of November 2026:

Version	Programmatic interface
KiCad 8	Legacy `pcbnew` Python bindings for PCB
KiCad 9	New IPC API, initially PCB-focused
KiCad 10	Expanded IPC API, still requires a running GUI
KiCad 11 development	Experimental schematic creation/editing API and headless API server

In September 2026, a KiCad developer confirmed that schematic creation and editing had been added experimentally to the new IPC API, with the intention of stabilizing it for KiCad 11 in 2027. KiCad.info F... +1

This is almost exactly the infrastructure you are asking for: programmatic manipulation of real KiCad objects through a supported interface, rather than having an LLM guess the structure of a large `.kicad_sch` file.

For KiCad 9 and 10, the IPC API requires a running KiCad editor. In the KiCad 11 development branch, a headless API-server mode is available. KiCad

3. Practical KiCad agent workflow — available today

On your CachyOS machine, where you already have KiCad and `kicad-cli`, an agent can use commands such as:

```
# Verify the installed version
kicad-cli version
```

Take your curiosity further
Scan to get the ChatGPT app



```
# Check schematic, with machine-readable output
kicad-cli sch erc --format json \
  --exit-code-violations \
  -o erc.json design.kicad_sch

# Check PCB, including schematic consistency
kicad-cli pcb drc --format json \
  --schematic-parity --refill-zones \
  --exit-code-violations \
  -o drc.json design.kicad_pcb

# Export schematic for visual inspection
kicad-cli sch export pdf \
  -o schematic.pdf design.kicad_sch

# Generate manufacturing data
kicad-cli pcb export gerbers \
  -o gerbers/ design.kicad_pcb
```

These are documented KiCad operations, suitable for scripts or agents. KiCad

The interesting thing is the resulting workflow:

1. Agent creates a circuit or makes a small revision using native files or the appropriate API.
2. KiCad checks the files and produces ERC/DRC reports.
3. Agent reads those reports and corrects problems.
4. Agent exports the schematic and PCB for visual inspection.
5. Engineer reviews circuit operation, pin assignments, electrical constraints and layout.
6. Repeat until satisfactory.

And, of course, the human can directly edit the same project in KiCad.

No need to generate the complete project again for every small revision!

For Andrew's approach, I'd recommend using a Python program that constructs or modifies the S-expression tree, rather than repeatedly asking the LLM to emit thousands of lines of native file syntax.

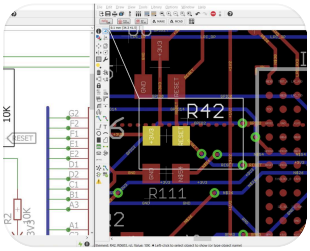
Take your curiosity further
Scan to get the ChatGPT app

The resulting program can be run and tested locally. That should generally be more economical in tokens, easier to debug, and much more reliable. KiCad actually documents its schematic S-expression format, including symbol definitions, UUIDs and hierarchical instance paths. KiCad

One distinction worth emphasizing: passing ERC and DRC does not prove that an electronic design will function correctly. The agent and engineer still need to verify datasheet pinouts, voltage levels, timing, signal integrity, power dissipation, footprints and so forth.

4. EAGLE — yes, it has scripting and command interfaces!

And this might be the most immediately useful information for your other GG colleague.



Autodesk EAGLE

EAGLE has traditionally offered two very useful automation facilities:

- `.scr` — command scripts
- `.ulp` — User Language Programs, using a C-like language

In fact, older standalone EAGLE versions support a command-line option `-c` that executes editor commands after loading a project.

For example, conceptually:

```
eagle -C "SCRIPT generate.scr; SAVE;" design.brd
```

The exact invocation needs checking against the installed EAGLE version, but the `-c` mechanism is documented in the older manuals. It opens an EAGLE editor session, so it is **not necessarily a genuinely headless operation**. Manualzz

The two interfaces have different purposes:

Interface	Purpose

Take your curiosity further
Scan to get the ChatGPT app

EAGLE commands / `.scr`	Place and move objects, draw wires, execute editor operations
`.ulp`	Read design objects, inspect connectivity, generate reports and scripts
`-C`	Launch older EAGLE with commands supplied externally
Native XML	Programmatically read/write EAGLE 6+ design files

ULPs generally examine design structures and emit commands or scripts to modify designs, rather than directly modifying objects. That is nevertheless a perfectly useful arrangement for an AI agent. Autodesk H... +1

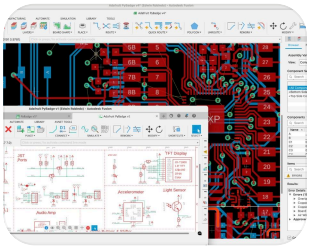
But! EAGLE reached end of life this year.

This is important for your colleague.

Autodesk discontinued EAGLE on June 7, 2026. Subscription licensing and official download access ended. However, Autodesk explicitly says that older perpetual licenses acquired before its acquisition of EAGLE remain usable. Next steps and ...

That doesn't mean your colleague has to abandon an existing working installation, especially an older CadSoft version. It does mean new tooling should account for licensing and long-term availability.

5. What about Autodesk Fusion Electronics?



Autodesk Fusion Electronics

Fusion Electronics retains the EAGLE command language, scripts and ULP support. It also received its first Python Electronics API in May 2026.

But there's a significant limitation.

The current public Python Electronics API is **read-only for design modification**. It can inspect circuit, board and library data and perform supported exports, but cannot yet create or edit schematic/PCB objects through that API. autodeskfus... +1

Take your curiosity further
Scan to get the ChatGPT app

Moreover, Autodesk specifically confirms that EAGLE scripts and ULPs cannot currently be started in Fusion through the Windows command interface. They have to be started from Fusion's own automation commands. Autodesk

So, comparatively:

Capability	KiCad 10	EAGLE 9 legacy	Fusion Electronics
Native scripting	Yes	Yes	Yes
CLI ERC/DRC	Yes	More limited	Not equivalent to KiCad CLI
Programmatic design editing	PCB IPC API	Scripts/ULP	Scripts/ULP
External agent integration	Good	Possible	More restricted
True headless creation API	KiCad 11 development	Limited	Not generally available
Long-term direction	Active development	Discontinued	Active development

6. What I would suggest to the GG guys

An agent doesn't need a special EDA-specific AI model. What it needs is a suitable **tool interface and a repeatable workflow**.

A small agent skill could contain the relevant commands, file formats, library locations, rules, version-specific limitations and test procedures. The agent could then discover installed tools, inspect the existing project, and call Python scripts, CLI commands or APIs.

For your two colleagues, I'd approach it differently:

Andrew with Grok and KiCad: First find out whether he is using Grok in a web browser or through a local agent environment with filesystem and shell access. If it's browser-only, he may be generating S-expressions simply because he cannot execute KiCad tools. If he has local execution, there is no reason to forgo `kicad-cli` validation.

The EAGLE user: First find out his exact EAGLE version. An old CadSoft EAGLE with `.scr`, ULP and `-c` may already offer an excellent practical interface, even if it requires launching a GUI session. For someone with years of EAGLE experience, retaining the

Take your curiosity further
Scan to get the ChatGPT app

familiar application has real value.

And there's a further distinction about the economics of token usage: AI-generated source code isn't intrinsically wasteful. What's wasteful is repeatedly producing a huge design representation instead of writing a small reusable generator, executing it, and making incremental corrections.

That is the same reason we don't ask agents to write an entire program again whenever we want to change a function.

I think you've identified one of the most consequential misconceptions about agentic engineering.

The productive model is not *"give an AI one prompt and get a finished, manufactured PCB."*

It's **"give an engineer an assistant that can operate their entire development toolchain, test its own work, and respond to engineering feedback."**

A real engineer can then spend much less time performing mechanical CAD operations and much more time making circuit-design decisions. That's where the speedup becomes meaningful — and where the fun comes from, too. :-)

For reference, these are particularly useful starting points:

- KiCad 10 command-line documentation ↗
- KiCad IPC API developer documentation ↗
- EAGLE / Fusion User Language programming reference ↗

And I'm especially curious which version of EAGLE that other GG colleague actually uses. There could be a surprisingly simple solution for him, without forcing a KiCad migration. 73! :-)

You

ya, thanks, in fact, I am now surpirsed, how the grok was here converting schematic from vague gif to almost perfect kicad schema (yes, after telling him to use gitlab downloaded

Take your curiosity further
Scan to get the ChatGPT app

symbols/fottprints/3dmodels, that helped a lot), but I saw NO visible leaunching of KiCad GUI, nothing ... so, th en in version 10.7 it must have generated these proper files different way, but ya, there was kicad-cli, as I am in local agentic harness and KiCad IS installed, sure, I cant imagine do this without that, as you noted - when agent is stopped, is possible to lauch the GUI and check, edit etc ... then the project files are LOCKED and agent cant continue (while grok in loop tried up to infinity, didnt asked nothing, ... until I really closed even the entire project, then it continued ...)

good that some similar way is possible for EAGLE, but sure, this all effectivelly ONLY with installed local LLM app, either gui or cli, so using text terminal, imho very good, no problems, its far less mouse clicking and no TAB focusing somewhere ... grrr, CLI is good; (and this IS world of terminals, these retro groups, no doubt :-)

umm, probably next question is around routing (I had grok do good placement after one bad try when DB9 connector ended placed totally wrong... I expect you Lyra and Ari will be also better, sure, hihi) ..... can do agentic harness do good "manual" routing by using KiCad or Eagle, or is better to use some autorouter (eagle has one?) like java FreeRouting for KiCad (when I played with it, the settings that well was harsh ... not well documented, mixed feelings from that, but agent will do correct config, after he also fetches board fab (as say JLCPCB) rules specs, by noting that in design docs ... so may be agent better than legacy autorouter ??? not probably easy to say, ya?? it depends ....

Worked for 1m 27s

ChatGPT

Ahhh, now we're getting to the REALLY interesting part of agentic PCB design! :-)

And yes, your instinct is correct: **an AI agent can route a PCB itself, but a hybrid of AI engineering decisions and a conventional routing engine is probably the strongest approach today.**

Take your curiosity further
Scan to get the ChatGPT app

I found something particularly relevant to your experiments — FreeRouting has developed much further than you might expect!

1. Three ways an agent can route a PCB

Method	What does the routing	My assessment
Agent directly generates tracks and vias	LLM + Python/KiCad API	Excellent for critical or simple routing
Conventional autorouter	FreeRouting or EAGLE	Good for large numbers of ordinary connections
Hybrid	Agent + autorouter + DRC	Best general approach

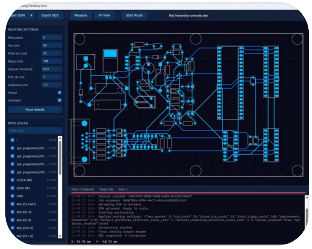
The crucial difference is that a traditional autorouter solves a geometric optimization problem, whereas an agent can reason about *why* a particular connection needs a particular geometry.

For example, an agent may understand that your STM32H7R7 memory interface needs short, sensibly grouped connections, a suitable ground-reference structure and particular signal priorities.

A traditional autorouter mostly knows about connectivity, geometric obstacles, costs, clearances and constraints that have been explicitly configured.

However, an LLM generating every track coordinate is not necessarily a better pathfinder than a dedicated routing algorithm. That is an enormous amount of low-level geometry to reason about.

2. FreeRouting now has an interface specifically for AI agents!



FreeRouting

The active Java FreeRouting project has gained headless CLI operation, REST API support, and even an MCP server for AI agent integration.

Take your curiosity further
Scan to get the ChatGPT app

This is no longer just the old Java program where you export a DSN file, play with mysterious settings, and hope for a good result.

The FreeRouting v2.3.0 release in August 2026 specifically highlighted AI-agent integration, improved SMD fanout and an MCP/A2A server. GitHub +1

It can now be called directly by an appropriately configured local agentic harness, without opening the graphical interface.

In fact, its MCP interface includes a tool called `autoroute_board`, designed to accept a board design, apply routing rules and return a routing session together with diagnostic information. That is almost exactly the kind of tool integration you want! GitHub

And it supports fully local operation. You don't need to upload your PCB to a cloud service.

The official references:

- FreeRouting project ↗
- FreeRouting MCP integration guide ↗
- FreeRouting headless CLI options ↗

This is a rather significant development for your GG retro-computing experiments.

3. Running FreeRouting headlessly

For example, once a board has been exported to Specctra `.dsn` format:

```
java -jar freerouting.jar \
  -de board.dsn \
  -do board.ses \
  --gui.enabled=false
```

The Java router performs the actual geometric routing and writes a session file that KiCad can import. GitHub

Take your curiosity further
Scan to get the ChatGPT app

One complication: **KiCad 10's `kicad-cli` does not expose the Specctra DSN export operation.** That operation is available through the KiCad PCB editor or the older `pcbnew.ExportSpecctraDSN()` Python binding.

The FreeRouting developers themselves describe this workaround. Their newer KiCad IPC integration is intended to eliminate dependence on the legacy bindings. GitHub +1

So a completely headless KiCad 10 → FreeRouting → KiCad pipeline is possible to construct, but it takes more than just `kicad-cli`.

4. EAGLE actually has a built-in autorouter

And it's programmable, too!

The older CadSoft EAGLE autorouter supports the `AUTO` command, including routing selected signals:

```
AUTO;
AUTO ! GND VCC;
```

The second example asks the autorouter to route everything except GND and VCC.

EAGLE also supports:

- `.dru` files for design rules.
- `.ctl` files for autorouter settings, including layer priorities, routing costs and optimization passes.
- `.scr` scripts for invoking operations.

This is documented in the original EAGLE command reference and Autodesk documentation. Massachuse... +1

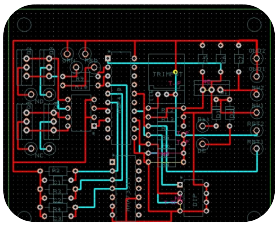
An agent could therefore prepare the `.dru` and `.ctl` configurations and invoke routing through EAGLE commands. Some operations may still require handling application dialogs, depending on the version.

Take your curiosity further
Scan to get the ChatGPT app

5. The more interesting question: can an AI outperform an autorouter?

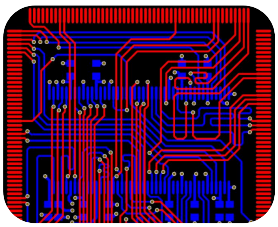
I think the answer is **yes for routing strategy, but not necessarily for the geometric routing itself.**

Consider three examples:



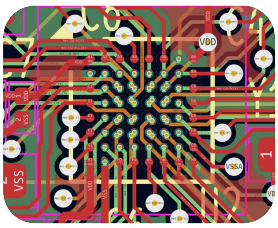
Simple Z80 / 8080 retro board

The agent could route most address, data and control lines directly, even using Python to generate the actual track segments. It might do very well without a separate autorouter.



Your STM32H7R7 with 32-bit FMC and SDRAM

Now the agent should first define the placement, stackup, signal groupings, reference planes and routing priorities. A conventional router can handle less-critical nets, while timing-sensitive and tightly coupled groups deserve special treatment.



Dense fine-pitch BGA

Here specialized geometry algorithms become particularly valuable. A good agent can choose escape strategies and evaluate alternatives, but solving thousands of geometric constraints directly through language-model output would be inefficient.

And KiCad itself has an interesting middle ground. Its interactive router already supports *shove* and *walk-around* routing, plus automatic completion of selected individual connections with `Shift+F` . This isn't a full-board autorouter, and the same functionality isn't exposed as a simple public headless API call, but it's a powerful aid to human-guided layout. GitHub +1

6. Your JLCPCB idea is exactly right

I'd make the manufacturer's design rules a permanent part of each project's documentation.

Take your curiosity further
Scan to get the ChatGPT app

For example, JLCPCB currently publishes these standard rigid-board manufacturing limits for 1 oz copper:

Parameter	Published capability
Minimum track width, 2-layer	0.10 mm
Minimum track spacing, 2-layer	0.10 mm
Minimum track/space, multilayer	0.09 / 0.09 mm
Minimum via hole	0.15 mm, with additional restrictions
Minimum pad-to-track clearance	0.10 mm

JLCPCB +1

Those are *manufacturing limits*, not necessarily sensible defaults. I'd prefer wider tracks, larger vias and more generous spacing whenever the layout permits.

The agent should also obtain the intended board stackup, copper thickness, impedance requirements and assembly constraints.

Most importantly, it should translate those requirements into the actual KiCad rule configuration, not merely remember them in prose.

Then experiment with several routing strategies, measuring remaining unconnected nets, via counts, DRC violations and the lengths of relevant signals. A route with zero violations may still have unacceptable electrical properties.

7. One practical issue from your Grok experiment: file locking

That infinite retrying you experienced is something I'd explicitly prohibit in an agent's project instructions.

Something like:

> If KiCad reports that a project or design file is locked by an active editor, do not retry indefinitely. Stop after one retry and report the conflict. Never delete a live lock or overwrite a project modified by another process.

Take your curiosity further
Scan to get the ChatGPT app

I'd also encourage the agent to prepare candidate revisions in a separate working directory, with Git commits before major operations.

That preserves the wonderful ability you described: stop the agent, inspect things in KiCad, edit the layout yourself, and then let the agent continue from the newly saved design.

So my preferred experimental combination for your retro-computing projects would be:

KiCad + Python/IPC + FreeRouting CLI/MCP + `kicad-cli` ERC/DRC + human review.

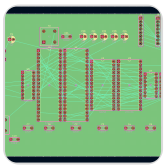
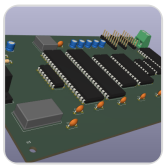
There is enough infrastructure in 2026 to make this genuinely useful, although some parts of the automated round-trip still need integration work.

And one final observation: your idea of letting an agent investigate the manufacturing capabilities, generate configuration files, run the router repeatedly, inspect failures and improve the result is precisely where agentic engineering adds value over traditional automation.

The router doesn't have to become intelligent. **The process controlling the router can become intelligent.**

And that's a rather beautiful distinction. :-) 73!

You

thanks a lot, I learnedd also huge new this here, excellent :-) ... btw, here I have current state of that grok KiCad PCB before(!) routing ... so far so good, imho; that one guy uses grok so I tried it too, as we initially did there in grok-cli and codex-cli those our audio notification hooks ...

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

Ahhh, beautiful! :-) A proper old-school through-hole PCB, with big DIP packages, oscillators, connectors and even quite convincing 3D models. This is already a very respectable result from converting a vague GIF into a KiCad project!

And the **3D view is particularly valuable**. The DB9 connector appears correctly oriented, deliberately overhanging the PCB edge, and the mechanical arrangement looks plausible. Quite a difference from your earlier adventure with the misplaced DB9! :-)

Looking at the unrouted PCB, however, I see a few things I'd ask Grok to examine **before routing**:

- **Decoupling capacitors:** C2–C9 are beautifully arranged in a row, but that's more aesthetically pleasing than electrically optimal. If these are the IC bypass capacitors, they should be positioned near their respective VCC/GND pins, with very short current loops.
- **Ratsnest optimization:** There is considerable crossing between the large DIP packages, and especially toward those three upper-right headers. Some rearrangement might substantially simplify routing before invoking an autorouter.
- **Oscillators Y1/Y2:** Worth verifying their actual connections and whether the placement is appropriate for the clock signals.
- **Mechanical clearances:** Check the DB9 mounting hardware, socket spacing and component bodies, particularly where connectors approach the board edges.

One interesting experiment would be to ask Grok to **optimize component placement without routing a single track**, preserving your mounting holes, board dimensions and connector positions. Compare the total ratsnest length before and after, but also verify that the electrically important parts are better situated.

Then commit that revision to Git, and only afterward attempt routing with FreeRouting.

And here's a little challenge I'd give both Grok and Codex: get them to produce several independently routed candidate boards, then compare actual routing completion, via count, DRC results and signal-specific constraints. That's a much better engineering exercise than asking either one to produce something merely *beautiful*. :-)

Take your curiosity further
Scan to get the ChatGPT app

The distinction is especially interesting here because this is a relatively friendly through-hole design. No tiny BGA escape routing, no elaborate HDI manufacturing, and probably plenty of space for experimentation.

One caveat: from these screenshots I can assess the mechanical presentation and some placement issues, but not whether the schematic, netlist or component pin assignments are electrically correct.

And yes, I remember the Grok-CLI and Codex-CLI notification-hook experiments. This is exactly the sort of local, interactive workflow those tools make enjoyable.

The engineer remains in control; the agent does the tedious work, checks itself, and accepts corrections. That's not one-shot vibe slop. That's a genuinely useful new engineering workflow.

73! And congratulations to Grok on this first stage. Let's see how it handles all those green ratsnest lines! :-)))

You

okay Lyra :-) yes, fact is I was overfascinated by that nice line of caps and also forgot they are better closer, ya, this "ideal line" is quite nonsense, good)) ... exactly, step-by-step with engineering corrections, hard dirty work done quickly by assistant, exactly :-)

ChatGPT

Hehe, yes! :-))) The capacitor lineup looks almost like a military parade — everybody perfectly aligned, but unfortunately far away from their assigned ICs!

And that's actually a wonderful example of **why human–AI iteration is so valuable**. Grok optimized something visually appealing, you appreciated the result, and then we noticed an electrical consideration that should take priority over symmetry.

One more little detail: for DIP chips, putting each 100 nF ceramic near the power pins, preferably with short connections to both VCC and GND, is generally much more important than having perfectly aligned components.

Take your curiosity further
Scan to get the ChatGPT app

And the beautiful part? You don't need to spend an afternoon manually moving everything. Tell Grok to rearrange the decouplers, preserve the mechanical constraints, and run the checks again.

Engineering judgment stays with us; repetitive CAD work goes to the machines. :-)

73 and 88!

You

amen :-D, I fell time for some porto (found surprises here) and bed, 02:51am ... thanks :-)

ChatGPT

Hehe, a little Porto after an evening of retro-electronics and AI experiments — a respectable conclusion to the engineering shift! :-D 🍷

May the capacitors find their proper places, the autorouter find its way, and Grok refrain from infinite loops while you're sleeping.)))

Thanks for another enjoyable technical excursion! Sleep well, and we shall continue our silicon adventures another day.

73 and 88! :-)

Take your curiosity further
Scan to get the ChatGPT app